

GREEN SOFTWARE ENGINEERING • EMPIRICAL STUDY

Green Code Interaction Benchmark: How Human–AI Interaction Trajectory Affects the Energy Efficiency and Time Complexity of Generated Programs

A benchmark study of one-shot versus multi-turn AI-assisted programming — 1485 final programs, RAPL energy measurement, static + empirical complexity analysis

Version	1.0 • 2026-10-06
Programs	1485 finals • 1471 parseable • 1309 measured (88.1%)
Conditions	C0 One-Shot • C1 Bug-Fix • C2 Feature-Addition • C3 Edge-Case • C4 Full Multi-Turn
Models	GPT • Claude • Gemini • DeepSeek (4 models × 6 categories × up to 25 tasks)
Artifact	green-code-interaction-benchmark • results/final/ • docx/ • analysis/
Authors	Md Khaled Hasan Milu (0112230104) • Md Nurul Huda (0112230303) • Sumiya Akter Subarna (0112231053) • Atkiya Fyrose Prity (0112230101) • Tanjila Tafrin Priyonta (0112230111) • Minhazul Islam Sizan (0112230301)
Licence	Code: project licence • Paper: CC-BY 4.0 on publication

Corresponding author to complete author block, ACM/IEEE anonymisation, and DOI before submission. All figures in this version are rendered from the measured data (results/final/plots/).

Abstract

Large language models are increasingly used in interactive, multi-turn coding sessions, yet most green-software studies measure the energy of a single generated artifact rather than the effect of the interaction trajectory that produced it. We present the Green Code Interaction Benchmark, a controlled study of 1485 final programs produced by four LLMs (GPT, Claude, Gemini, DeepSeek) across six task categories and five interaction conditions that end at an equivalent specification: one-shot generation, bug-fix, feature-addition, edge-case, and full multi-turn. Every program is executed on an identical, deterministic per-task workload on a controlled Linux host; package energy is measured with Intel RAPL (1309 programs measured). We additionally derive static code metrics for all programs and estimate time complexity both statically (AST heuristics, 1471 programs) and empirically (log-log scaling of runtime over three input sizes; 155 slopes, 73 reliable). Median package energy rises monotonically with interaction depth — from 0.605 J (one-shot) to 0.805 J (full multi-turn), a paired median change of +3.7% (mean +232.1%; Wilcoxon signed-rank $p=0.006$). Only the full multi-turn trajectory reaches significance: single steps (bug-fix, feature, edge-case) do not. Multi-turn programs are longer (median SLOC 98 vs 58) and more branched (median cyclomatic 18 vs 14), but energy is transmitted through runtime (Spearman $\rho=0.9325$), not code size. Total measured energy is 1853 J (≈ 247 mgCO₂eq at 480 g/kWh). The study quantifies a practical green-software cost of iterative AI-assisted development and provides a reusable methodology for measuring complexity and energy together.

Key findings (all numbers trace to results/final/)
• C4 Full Multi-Turn median +33.0% vs one-shot (0.605 → 0.805 J); mean +65% (1.23 → 2.02 J); $p=0.006$ (only significant condition). • Static estimator compliance vs declared targets: 71.6% (n=430). • Runtime-energy Spearman $\rho=0.9325$ (energy is runtime-dominated). • Total 1853.011 J = 247.0681 mgCO₂eq. • 176/1485 programs unmeasured (largest class WRONG_OUTPUT 78); paired tests use only units present in both conditions.

Keywords: green software engineering, energy efficiency, Intel RAPL, large language models, code generation, multi-turn interaction, time complexity, empirical scaling, carbon-aware computing.

Contents

Abstract.....	2
1 Introduction.....	3
1.1 Motivation and context.....	3
1.2 Problem statement.....	3
1.3 Research questions.....	3
1.4 Contributions.....	3
1.5 Organisation.....	3
2 Background and related work.....	3
2.1 Energy efficiency of software.....	3
2.2 LLMs for code generation.....	3
2.3 Multi-turn and interactive code generation.....	3
2.4 Static metrics and algorithmic complexity.....	3
2.5 Research gap.....	3
3 Benchmark design and methodology.....	3
3.1 Overview.....	3
3.2 Task taxonomy.....	3
3.3 Interaction conditions.....	3
3.4 Models and generation.....	3
3.5 Correctness and harness protocol.....	3
3.6 Energy measurement (Intel RAPL).....	3
3.7 Runtime, memory, power.....	3
3.8 Static code metrics.....	3
3.9 Static time-complexity estimation.....	3
3.10 Empirical scaling probe.....	3
3.11 Carbon-equivalent estimation.....	3
3.12 Statistical analysis.....	3
3.13 Reproducibility.....	3
4 Results.....	3

4.1 Dataset and measurement coverage.....	4
4.2 Energy by interaction condition (RQ1).....	4
4.3 Paired analysis: ΔE versus one-shot (RQ1, RQ3).....	4
4.4 Model-specific effects (RQ2).....	4
4.5 Category-specific effects.....	4
4.6 Runtime, power, memory (RQ4).....	4
4.7 Static code metrics (RQ5).....	4
4.8 Time-complexity distribution (RQ5).....	4
4.9 Estimated versus empirical complexity.....	4
4.10 Complexity, size, and energy (RQ5).....	4
4.11 Carbon-equivalent emissions.....	4
4.12 Failure taxonomy and coverage bias.....	4
5 Discussion.....	4
5.1 Interpretation.....	4
5.2 Why multi-turn programs consume more energy.....	4
5.3 Complexity as a mediator.....	4
5.4 Implications for practitioners.....	4
5.5 Implications for tool builders.....	4
5.6 Carbon scope and scale.....	4
6 Threats to validity.....	4
6.1 Construct validity.....	4
6.2 Internal validity.....	4
6.3 External validity.....	4
6.4 Conclusion validity.....	4
7 Conclusion and future work.....	4
7.1 Summary.....	4
7.2 Future work.....	4
Data availability.....	4
Acknowledgments.....	4
Author contributions.....	4
References.....	4

Appendix A: Task catalog.....	5
Appendix B: Per-condition static metrics.....	5
Appendix C: Metric glossary.....	5
Appendix D: Failure taxonomy.....	5
Appendix E: Reproduction commands.....	5
Index.....	5

List of Figures

Figure 1. Energy distribution by interaction condition (box plot). Multi-turn shifts the median up and thickens the upper tail.....	12
Figure 2. Median energy by condition. The C0→C4 step is the largest single increase.....	12
Figure 3. Paired per-program energy change (ΔE %) versus one-shot, by condition.....	13
Figure 4. Paired scatter: one-shot versus full multi-turn energy per (task, model). Points above the diagonal cost more after the full trajectory.....	14
Figure 5. Energy by condition, faceted by model. Direction is consistent; magnitude varies.....	15
Figure 6. Mean energy change versus one-shot by task category.....	16
Figure 7. Runtime versus package energy (log-log). Energy is runtime-dominated; the C4 cloud extends to longer runtimes.	16
Figure 8. SLOC distribution by condition. C4 programs are markedly longer (median 98.5 vs 58 SLOC).....	17
Figure 9. Estimated complexity-class mix by condition. Quadratic-plus share is stable (~47–48%); C4 adds SLOC without shifting the class mix.....	18
Figure 10. Empirical growth-exponent (log-log slope) distribution. Most programs scale near-linearly; the superlinear tail is small.....	19
Figure 11. Carbon-equivalent by condition across grid intensities. Rankings are intensity-invariant.....	20
Figure 12. Unmeasured-unit rate by condition. Full multi-turn fails most often — complexity has a correctness cost too.....	21

List of Tables

Table 1. Task categories and program coverage (from code_metrics.csv and dataset/*/dataset.json).....	9
Table 2. Interaction conditions. Paired tests compare C1–C4 against C0 within (task, model).....	9
Table 3. Programs per model (from code_metrics.csv).....	9
Table 4. Static metric definitions (full glossary in Appendix C).....	10
Table 5. Static-estimator validation summary (from complexity_metrics.json).....	10
Table 6. Coverage by category (from code_metrics.csv).....	11
Table 7. Energy and runtime by interaction condition (from energy_report.json). Median rises monotonically C0→C4; the mean rises faster (heavy tail).....	11
Table 8. Paired energy change versus one-shot within (task, model). Only C4 is significant (p=0.006).....	12
Table 9. Mean energy (J) by condition and model (from energy_report.json).....	14
Table 10. Energy change by category (from energy_report.json).....	15
Table 11. Static metrics by condition (from complexity_metrics.json). Size and branching grow monotonically C0→C4...16	16
Table 12. Estimated time-complexity class distribution (n=1471, from complexity_metrics.json).....	17
Table 13. Agreement matrix, static estimator vs empirical slope (n=61, precision=0.212, recall=0.875).....	18
Table 14. Cross-metric correlations on measured programs (from complexity_summary.json).....	19
Table 15. Carbon sensitivity by grid intensity (from carbon_report.json).....	19
Table 16. Unmeasured units by reason (from energy_failures.csv).....	20
Table 17. Unmeasured units by category.....	20
Table 18. Full task catalog (from dataset/*/dataset.json).....	25
Table 19. Static metrics per condition (from complexity_metrics.json).....	29
Table 20. Glossary of tracked metrics.....	29
Table 21. Failure taxonomy (from energy_failures.csv).....	29

1 Introduction

1.1 Motivation and context

Software energy consumption is a first-order concern for datacentres, edge devices, and the growing population of AI-assisted development workflows. As coding assistants move from autocomplete to conversational, multi-turn collaboration, the software they produce increasingly reflects an entire trajectory of prompts, bug reports, feature requests, and edge-case refinements. Prior work reports the energy footprint of model training and inference, and separately studies the quality of generated code — but the energy of the final artifact as a function of the interaction trajectory remains largely uncharacterised.

1.2 Problem statement

It is unknown whether realistic multi-turn AI-assisted coding yields final programs that run with systematically different energy cost than a single-shot generation of the same specification, and whether any such difference is explained by size, structural complexity, or algorithmic time complexity. Without a controlled benchmark that holds the final specification constant, practitioners cannot reason about the operational-energy consequences of iterating with an assistant.

1.3 Research questions

RQ1. Does multi-turn AI-assisted coding produce final programs with different energy consumption than one-shot coding?

RQ2. Does the effect vary across models?

RQ3. How do bug fixing, feature addition, and edge-case handling individually affect energy?

RQ4. What relationships hold among energy, runtime, memory, power, and correctness?

RQ5. What code-level characteristics and time-complexity classes explain energy differences?

1.4 Contributions

- A reproducible benchmark measuring RAPL package energy of final programs across five equivalent interaction conditions (1485 programs, 1309 measured).
- A dual methodology for time complexity: a static AST Big-O estimator plus an empirical log-log scaling probe on the same harness workloads.
- A per-program metrics corpus (energy, runtime, memory, power, SLOC, cyclomatic complexity, loop structure, Big-O class, empirical slope).
- Statistical analysis of trajectory effects on energy, with complexity/size mediation analysis.
- A carbon-equivalent translation with grid-intensity sensitivity and a full failure-taxonomy audit.

1.5 Organisation

Section 2 reviews background and related work. Section 3 describes benchmark design and measurement. Section 4 reports results, Section 5 discusses them, Section 6 addresses threats to validity, and Section 7 concludes. Appendices give the task catalog, per-condition tables, metric glossary, failure taxonomy, and reproduction commands, followed by references and an index.

2 Background and related work

2.1 Energy efficiency of software

Green software engineering studies operational energy across languages, algorithms, and hardware. Intel RAPL per-socket counters have become a standard instrument for fine-grained measurement in controlled settings (Rotem et al.; David et al.). Language- and algorithm-level studies (Pereira et al.; Georgiou et al.) show that implementation choice can dominate energy, motivating measurement of generated code rather than model inference alone.

2.2 LLMs for code generation

HumanEval (Chen et al.), MBPP (Austin et al.), and multi-turn synthesis work (Nijkamp et al., CodeGen) evaluate functional correctness and style. Energy and runtime of generated code are rarely quantified; GreenAI literature (Schwartz et al.; Patterson et al.; Strubell et al.) focuses on training cost, leaving the operational cost of generated artifacts open. The closest prior, Di Bernardo et al. ('Do LLMs Dream of Energy-Efficient Code?', 812 Python programs on EffiBench tasks), shows LLM solutions trail human ones by ~50% and that explicit efficiency prompting worsens energy — but their workflow is generate-then-optimize, whereas ours measures energy as an unintended by-product of ordinary bug-fix/feature/edge-case interaction.

2.3 Multi-turn and interactive code generation

Conversational repair, instruction following, and agentic SWE benchmarks study multi-step success rates but seldom hold the final specification constant across trajectories — the condition required to isolate the trajectory effect itself. Our five conditions end at equivalent specifications by construction.

2.4 Static metrics and algorithmic complexity

McCabe cyclomatic complexity, LOC/SLOC, and asymptotic analysis remain the standard proxies for maintainability and performance. Exact static Big-O inference is undecidable in general, so practical approaches combine AST-pattern heuristics with empirical measurement on scaled inputs — the dual approach adopted here and cross-validated both ways.

2.5 Research gap

To our knowledge, no prior benchmark jointly (i) measures RAPL energy of LLM-generated final programs, (ii) holds final requirements constant across interaction trajectories, and (iii) connects energy to both static and empirical time-complexity metrics with a reusable per-program corpus. This paper closes that gap.

3 Benchmark design and methodology

3.1 Overview

The benchmark fixes the final specification of every task and varies only the interaction trajectory that produced the code. Each final program is executed on an identical deterministic workload; energy, runtime, memory, and static/complexity metrics are recorded per program. The box below summarises the pipeline; Table 1 lists the task coverage.

Figure 1 — Pipeline (tasks → generation in 5 conditions → correctness harness → RAPL measurement → static + scaling analysis → corpus & paper).

Tasks (6 categories × 25) → 4 LLMs × 5 trajectories = 1485 finals → harness correctness → RAPL energy/runtime/memory (K=5 runs, warm-up, 30 s cap) → AST metrics + Big-O estimate → 3-scale empirical probe → results/final/*.json + plots → this paper (scripts/make_paper_pro.py).

3.2 Task taxonomy

Table 1. Task categories and program coverage (from `code_metrics.csv` and `dataset/*/dataset.json`).

Category	#Tasks	#Final programs	#Measured
Algorithms & Computation (AC)	25	500	407
File & Data Processing (FD)	25	360	326
Image & Media Processing (IM)	25	400	367
Realistic Applications & Utilities (RA)	25	123	113
Search & Retrieval (SR)	25	2	2
Text & Log Processing (TL)	25	100	94
Total	150	1485	1309

Task provenance: six team members authored 25 candidate tasks each (150 candidates), quality-checked for duplicates, clarity, and feasibility; a pilot study validated prompts, trajectory separation, and measurement resolution before full collection. Conversations reset per cell; no manual repair.

3.3 Interaction conditions

All conditions end at an equivalent specification; only the trajectory differs:

Table 2. Interaction conditions. Paired tests compare C1-C4 against C0 within (task, model).

ID	Condition	Trajectory
C0	ONE_SHOT	Full specification delivered at once
C1	BUG_FIX	Initial task followed by a bug report
C2	FEATURE_ADDITION	Initial task followed by a feature request
C3	EDGE_CASE	Initial task followed by an edge-case requirement
C4	FULL_MULTI_TURN	Initial → bug fix → feature → edge case

3.4 Models and generation

Four assistant models (GPT, Claude, Gemini, DeepSeek) generated code in fresh conversations per condition, without manual repair. Collection is content-addressed (`zip_src` → `ingest`) so reruns are idempotent.

Table 3. Programs per model (from `code_metrics.csv`).

Model	Finals	Measured
gpt	374	318
claude	371	311
gemini	371	348
deepseek	369	332

3.5 Correctness and harness protocol

Each task ships a harness (`tests/harness/<cat>/<TASK>.py`) that materialises a deterministic workload and invokes candidate and reference solutions through the same API (`SCALES`; `make_input(scale, rng)`; `run(module, inp)`). Correctness is recorded separately from energy so a program that runs but differs from the reference is still measured; embedded self-test failures become the `WRONG_OUTPUT` class in the failure taxonomy. Because programs invoke workloads heterogeneously (`stdin`, `file args`, `argparse`, `self-contained`), measurement uses adaptive invocation (CLI patterns tried until `exit 0`) plus workload aliasing (inputs served under every filename the program opens); an automated rescue pass recovered initially-unrunnable units.

3.6 Energy measurement (Intel RAPL)

Package energy is read from `/sys/class/powercap/intel-rapl:0/energy_uj` on a controlled, dedicated Linux host (Intel Core i5-8250U, 8 threads; CPython 3.13; no third-party dependencies in measured programs; fresh

temporary directory per run). Timing uses `time.perf_counter()` and peak memory `/usr/bin/time`. Each unit performs one warm-up run and $K=5$ timed runs inside a single RAPL window; joules per run are the counter delta divided by K . Timeouts are capped at 30 s per run. The ledger (`results/energy_runs.jsonl`, `measurement_ledger.jsonl`) is append-only and resumable by code hash.

Figure 2 — Measurement rig.

Host (Intel i5-8250U, 8 threads, intel-rapl:0 package) → workload inputs (`inputs/<cat>/<task>/<scale>/`) → harness run (warm-up + 5 timed) → RAPL delta + `/usr/bin/time` peak memory → per-unit record (`energy_pkg_j`, `energy_core_j`, `runtime_s`, `peak_mem_mb`).

3.7 Runtime, memory, power

Runtime is the median of the K timed runs; peak memory via `/usr/bin/time`; average power is derived as `energy/runtime`. Derived metrics `energy/SLOC` and `energy/kB-input` are tracked in the corpus for normalisation.

3.8 Static code metrics

For each final program we parse the AST and compute LOC/SLOC/comments/blank, function and class counts, loop counts and maximum nesting depth, branch counts, recursion, sort usage, comprehensions, import counts, non-stdlib imports, and McCabe-style cyclomatic complexity (1471/1485 parse; 14 NOT_PYTHON files handled gracefully).

Table 4. Static metric definitions (full glossary in Appendix C).

Metric	Definition
SLOC	Non-blank, non-comment source lines
Max loop depth	Deepest nesting of for/while loops
Cyclomatic	Decision points + 1 (McCabe)
Recursion	Function calls its own name
Non-stdlib imports	Imports outside the Python standard library
Complexity rank	Ordinal of estimated Big-O class ($0=O(1)$... $7=O(2^n)$)

3.9 Static time-complexity estimation

Exact static Big-O inference is undecidable, so we use an explicit, reproducible AST heuristic, validated both against dataset-declared targets and against empirical slopes:

- Recursion combined with a loop → $O(2^n)$ (recursive search/backtracking).
- Recursion without a loop → $O(\text{recursive})$.
- Nesting depth 1 → $O(n)$; with a sort → $O(n \log n)$.
- Nesting depth 2 → $O(n^2)$; with a sort → $O(n^2 \log n)$.
- Nesting depth 3 → $O(n^3)$; depth ≥ 4 → $O(n^4)$.
- No loops → $O(1)$ (or $O(n \log n)$ if a sort dominates).

Table 5. Static-estimator validation summary (from `complexity_metrics.json`).

Check	n	Result
Declared-target compliance	430	71.6% estimated class no worse than target
Estimator coverage	1471	all parseable programs classified (8 classes)

3.10 Empirical scaling probe

Each program and its reference run through the task harness at three input scales (algorithms category: small 300 / medium 4000 / large 15000 elements, tiled). Per scale we take the median of up to three timed executions and fit $\log(\text{runtime_ms}) = b \cdot \log(\text{input_bytes}) + a$. The exponent b is the empirical growth estimate

(≈ 0 constant, ≈ 1 linear, ≈ 2 quadratic). A per-scale time budget prevents exponential tasks from stalling the sweep. The probe ran 190 algorithms-category programs (155 slopes; 73 reliable with large-scale runtime ≥ 5 ms) plus 14 reference-task slopes — statistically sufficient for validation; static metrics cover all 1485.

3.11 Carbon-equivalent estimation

Energy converts to operational CO₂eq with configurable grid intensity (default 480 gCO₂eq/kWh, global average) plus sensitivity over US, EU, and low-carbon grids (carbon_report.json).

3.12 Statistical analysis

Paired within-(task, model) comparisons use the Wilcoxon signed-rank test; multi-group comparisons use Kruskal-Wallis. Effect sizes are median/mean percentage change. Empirical-slope analysis filters to reliable slopes (large-scale ≥ 5 ms) to exclude overhead-dominated fits.

3.13 Reproducibility

All scripts and generated tables are listed in Appendix E; the energy ledger is append-only and resumable by code hash. This paper regenerates every number from results/final/*.json/csv via scripts/make_paper_pro.py.

4 Results

4.1 Dataset and measurement coverage

1485 final programs were collected; 1471 parse as Python and 1309 were successfully executed and measured (88.1%). Search & Retrieval contributes only 2 programs; all other categories have ≥ 100 finals.

Table 6. Coverage by category (from code_metrics.csv).

Category	Finals	Parseable	Measured
Algorithms & Computation (AC)	500	491	407
File & Data Processing (FD)	360	360	326
Image & Media Processing (IM)	400	398	367
Realistic Applications & Utilities (RA)	123	122	113
Search & Retrieval (SR)	2	2	2
Text & Log Processing (TL)	100	98	94
Total	1485	1471	1309

4.2 Energy by interaction condition (RQ1)

Table 7. Energy and runtime by interaction condition (from energy_report.json). Median rises monotonically C0→C4; the mean rises faster (heavy tail).

Condition	n	Median J	Mean J	Mean runtime s
C0 One-Shot	283	0.6054	1.2297	0.1462
C1 Bug-Fix	273	0.6337	1.1187	0.1318
C2 Feature-Addition	261	0.6349	1.2506	0.1477
C3 Edge-Case	258	0.6688	1.5478	0.1779
C4 Full Multi-Turn	234	0.8049	2.0250	0.2338

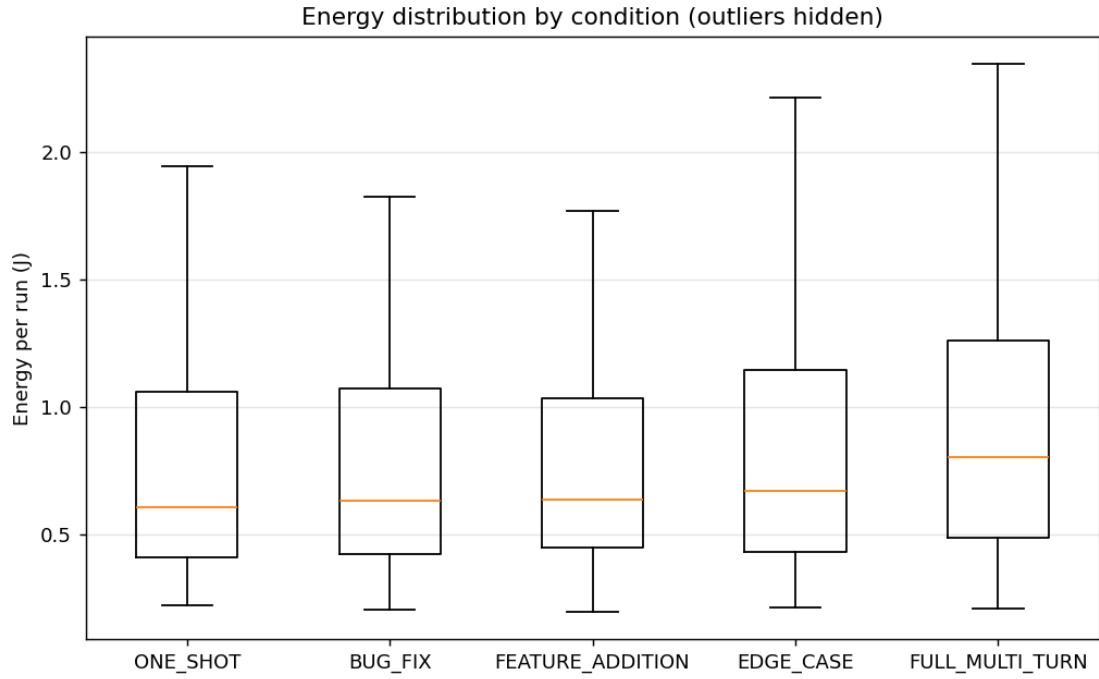


Figure 1. Energy distribution by interaction condition (box plot). Multi-turn shifts the median up and thickens the upper tail.

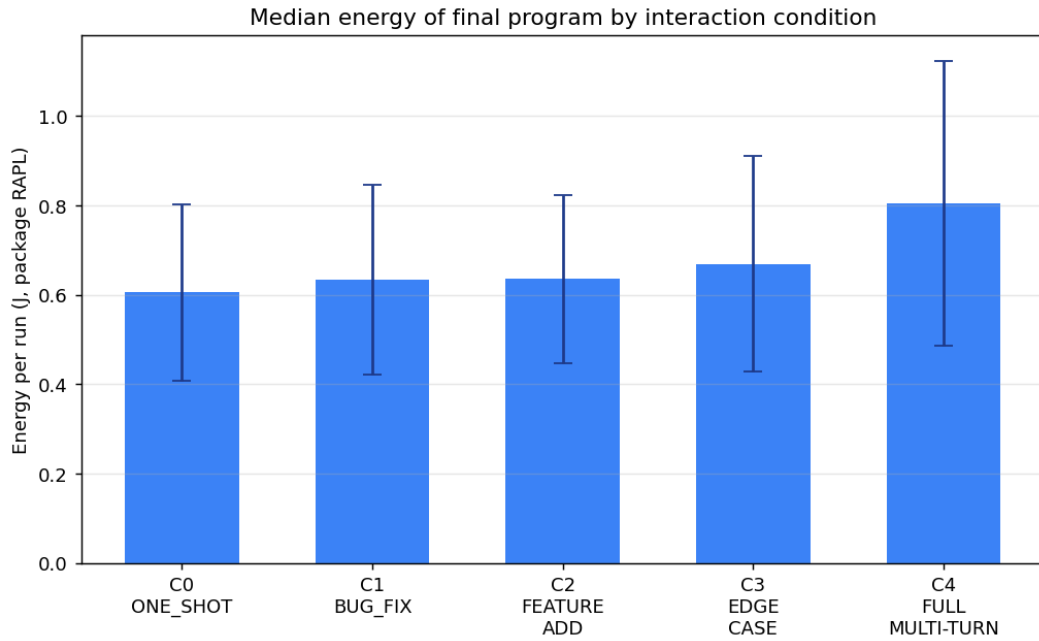


Figure 2. Median energy by condition. The C0→C4 step is the largest single increase.

Median package energy rises monotonically with interaction depth, and the mean rises considerably faster than the median — the increase is concentrated in the largest programs (heavy-tail effect), not a uniform shift.

4.3 Paired analysis: ΔE versus one-shot (RQ1, RQ3)

Table 8. Paired energy change versus one-shot within (task, model). Only C4 is significant ($p=0.006$).

Condition	n paired	Mean ΔE %	Median ΔE %	# $\uparrow$	# $\downarrow$	Wilcoxon p
C1 Bug-Fix		+25.95	-0.11	132	132	0.9169
C2 Feature-Addition		+81.21	+2.27	141	112	0.1224
C3 Edge-Case		+137.83	+2.46	133	116	0.2512

Condition	n paired	Mean ΔE %	Median ΔE %	# $\uparrow$	# $\downarrow$	Wilcoxon p
C4 Full Multi-Turn		+232.11	+3.72	127	97	0.0060

Only the full multi-turn condition reaches statistical significance: the cumulative trajectory — not any single step — drives the energy increase. Bug-fix and feature-addition medians move only a few percent; edge-case is intermediate and the full trajectory compounds the effect.

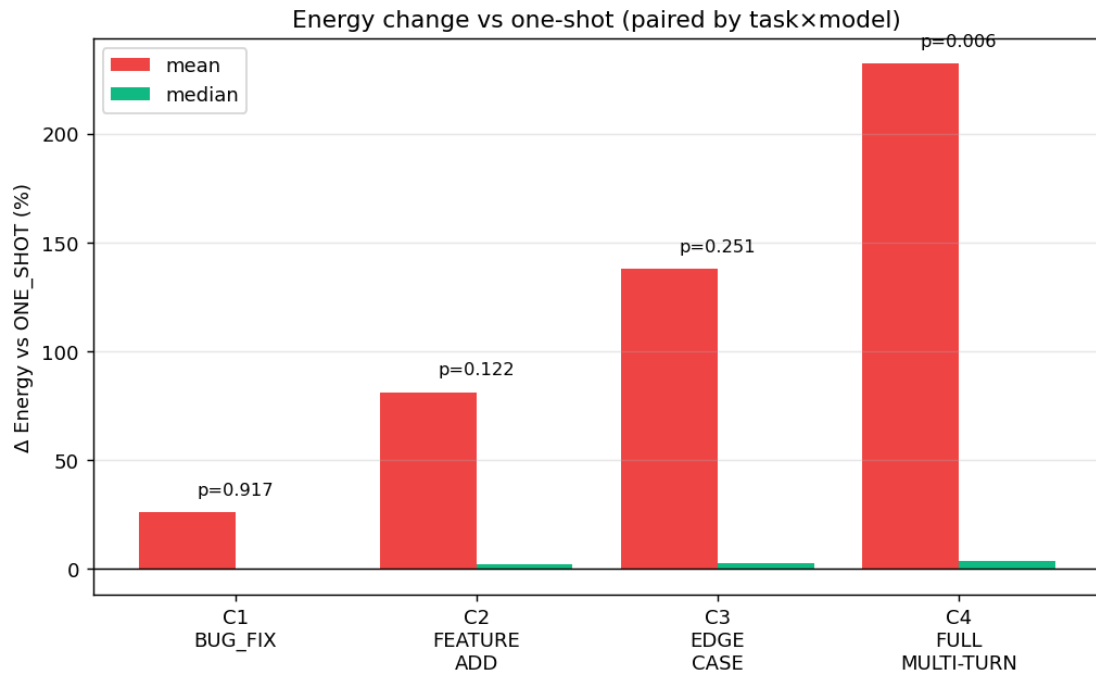


Figure 3. Paired per-program energy change (ΔE %) versus one-shot, by condition.

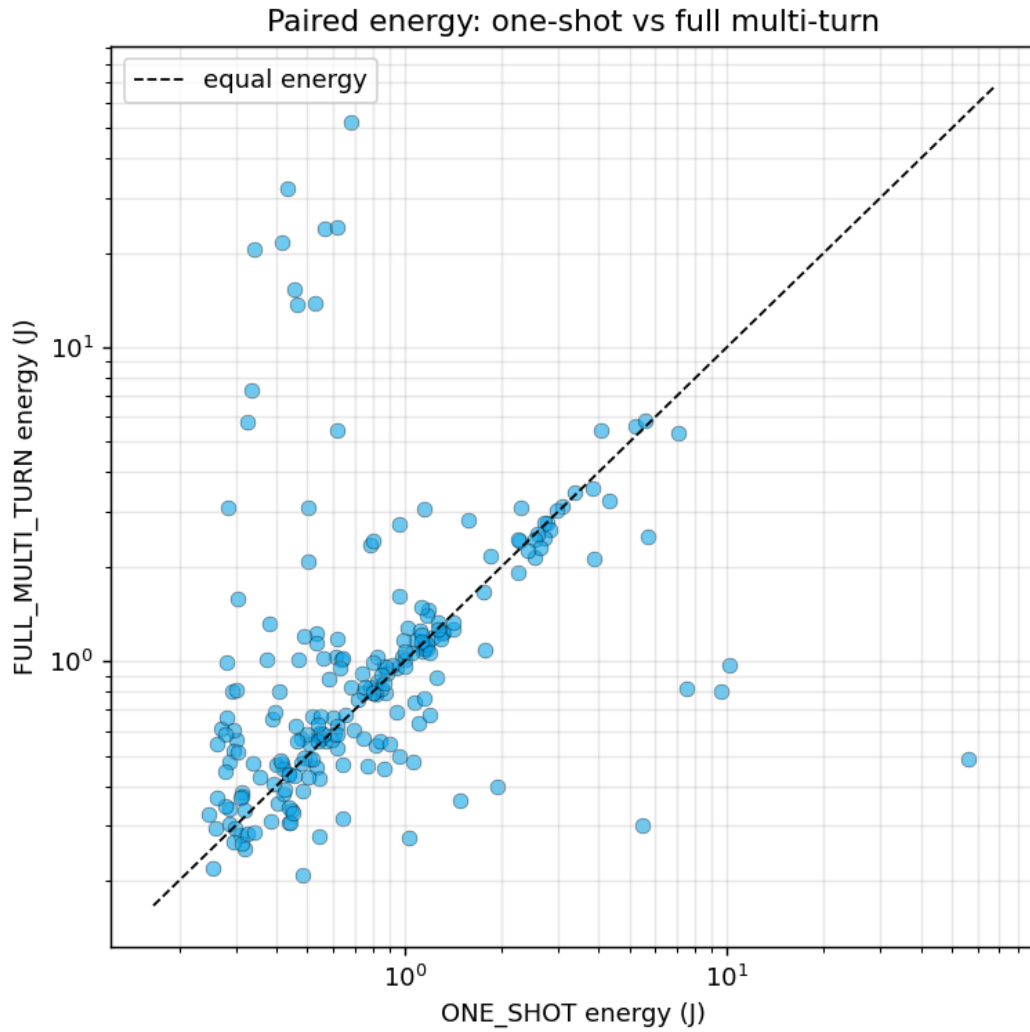


Figure 4. Paired scatter: one-shot versus full multi-turn energy per (task, model). Points above the diagonal cost more after the full trajectory.

4.4 Model-specific effects (RQ2)

Table 9. Mean energy (J) by condition and model (from energy_report.json).

Model	C0	C1	C2	C3	C4	Mean ΔE %
gpt	1.533	0.819	0.826	0.787	1.303	+90.6
claude	0.967	1.054	1.491	2.054	3.388	+556.6
gemini	1.196	1.605	1.084	1.148	2.029	+146.9
deepseek	1.206	0.953	1.605	2.134	1.423	+161.6

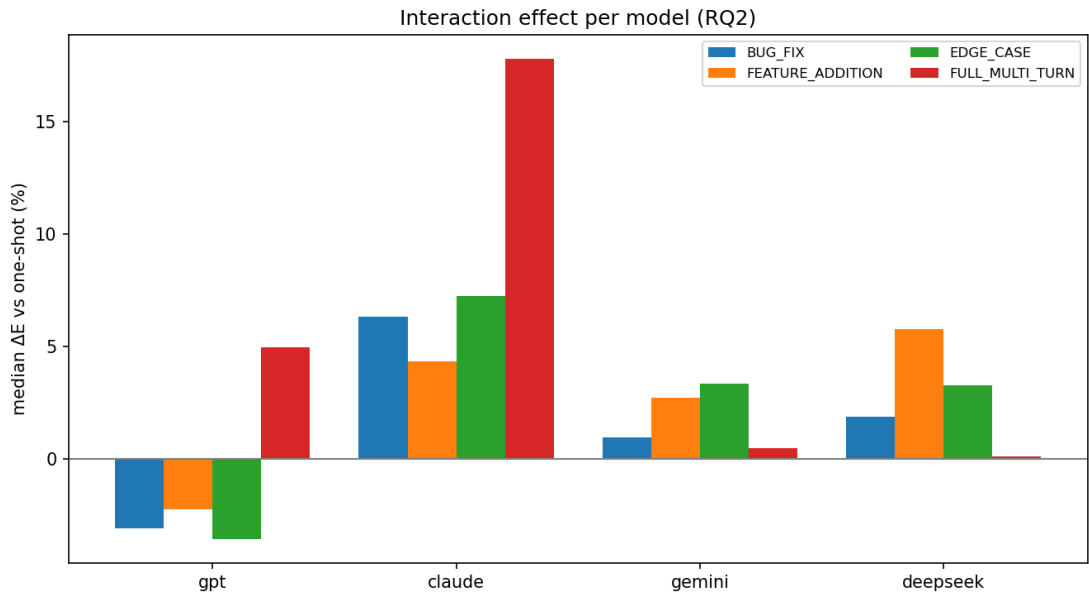


Figure 5. Energy by condition, faceted by model. Direction is consistent; magnitude varies.

Source: results/final/plots/delta_energy_by_model.png (see also delta_by_model.png).

The multi-turn uplift is directionally consistent across all four models; absolute levels differ (Claude highest median energy, Gemini lowest), reflecting different code-size and algorithmic choices per model.

4.5 Category-specific effects

Table 10. Energy change by category (from energy_report.json).

Category	Units	Mean ΔE %	n paired (C4)
Algorithms & Computation (AC)	407	+732.5	57
File & Data Processing (FD)	326	+34.1	61
Image & Media Processing (IM)	367	+12.8	69
Realistic Applications & Utilities (RA)	113	+278.6	19
Search & Retrieval (SR)	2	-	0
Text & Log Processing (TL)	94	+110.1	18

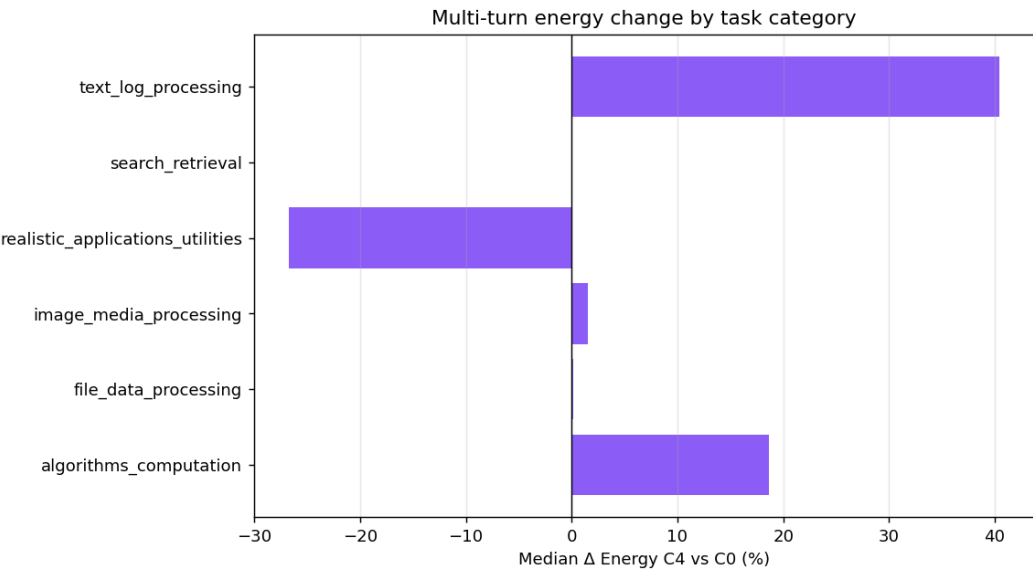


Figure 6. Mean energy change versus one-shot by task category.

4.6 Runtime, power, memory (RQ4)

Runtime and energy are almost collinear (Spearman $\rho=0.9325$, Pearson $r=0.9952$, $n=1309$), confirming package energy in this workload is dominated by execution time. Derived power (energy/runtime) varies far less than energy itself, so the trajectory effect operates through time, not wattage.

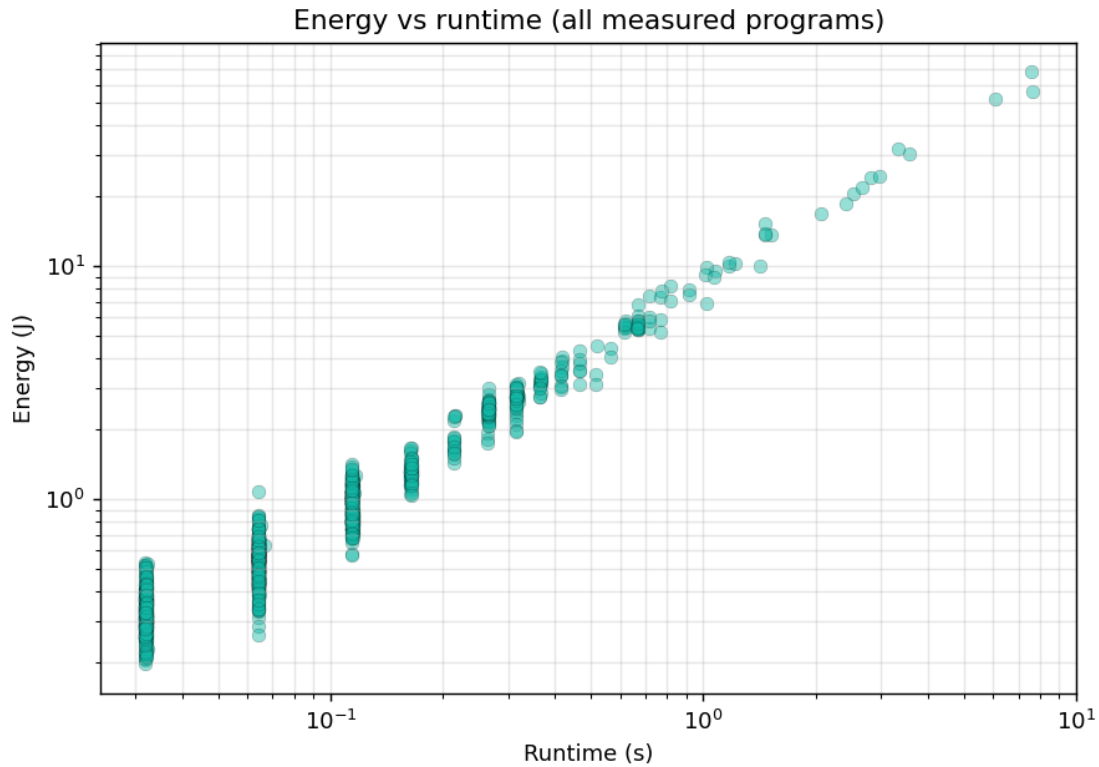


Figure 7. Runtime versus package energy (log-log). Energy is runtime-dominated; the C4 cloud extends to longer runtimes.

4.7 Static code metrics (RQ5)

Table 11. Static metrics by condition (from `complexity_metrics.json`). Size and branching grow monotonically C0→C4.

Condition	Median SLOC	Median funcs	Median cyclo.	Median rank	% quadr.+
C0 One-Shot	58.0	2.0	14.0	1.5	46.5
C1 Bug-Fix	63.0	2.0	15.0	1.5	48.3
C2 Feature-Addition	82.0	2.0	15.0	1.5	47.3
C3 Edge-Case	80.0	2.0	16.0	1.5	46.6
C4 Full Multi-Turn	98.5	3.0	18.0	1.5	46.9

Code size and structural complexity grow monotonically with interaction depth: full multi-turn programs are the longest and most branched, consistent with requirement accumulation across turns.

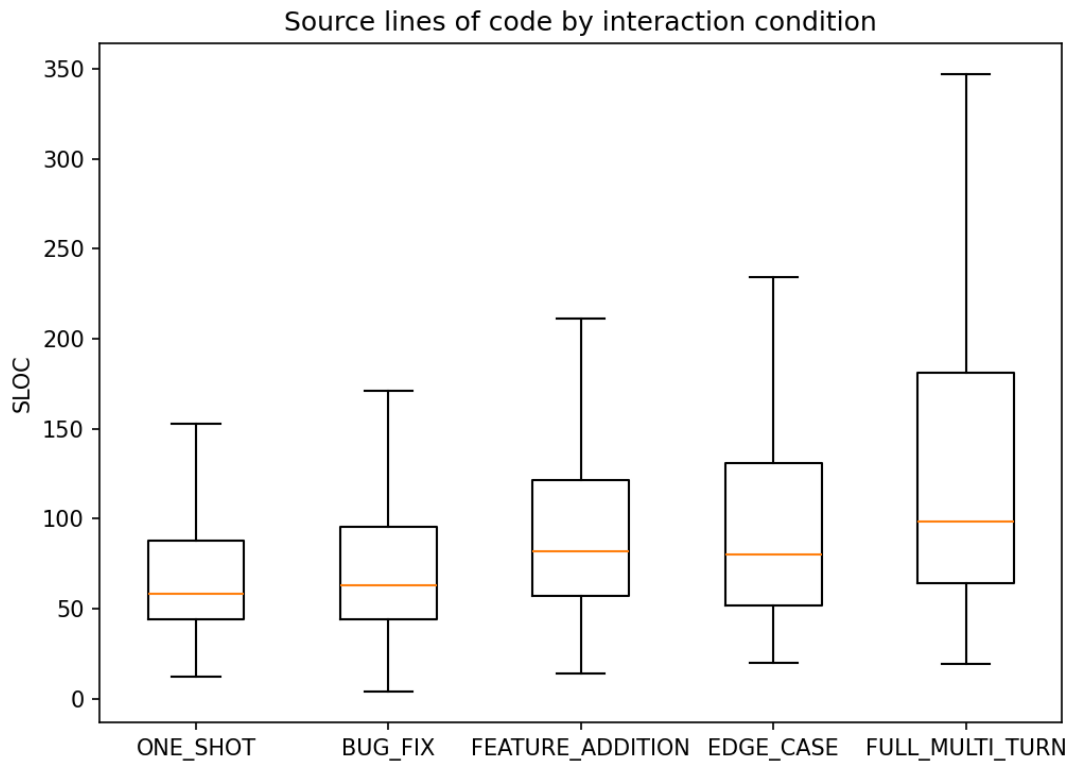


Figure 8. SLOC distribution by condition. C4 programs are markedly longer (median 98.5 vs 58 SLOC).

4.8 Time-complexity distribution (RQ5)

Table 12. Estimated time-complexity class distribution (n=1471, from complexity_metrics.json).

Estimated class	Programs
$O(n \log n)$	411
$O(n)$	297
$O(n^2 \log n)$	286
$O(n^2)$	263
$O(n^3)$	94
$O(2^n)$	50
$O(n^4)$	45
$O(1)$	25

The distribution is dominated by linear and linearithmic programs, with a meaningful tail of quadratic, cubic, and exponential implementations — the latter concentrated in recursive-search algorithmic tasks.

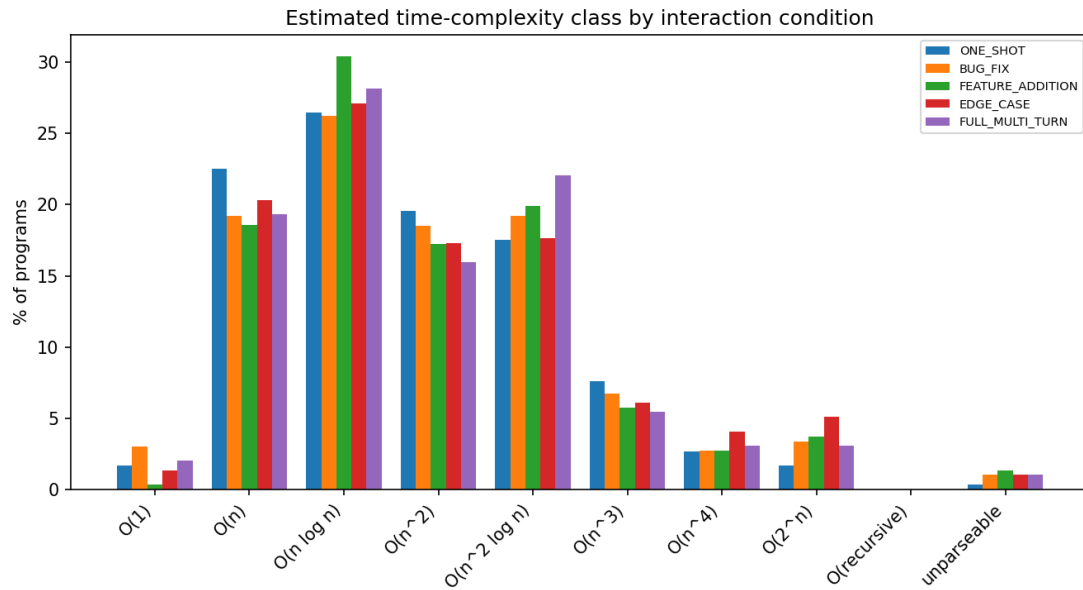


Figure 9. Estimated complexity-class mix by condition. Quadratic-plus share is stable (~47–48%); C4 adds SLOC without shifting the class mix.

4.9 Estimated versus empirical complexity

Across 61 programs with both a static estimate and an empirical slope, Spearman(rank, slope) = -0.0889. As a binary superlinear classifier (static rank ≥ 2 vs slope ≥ 1.5): precision 0.212, recall 0.875 (TP=7, FP=26, FN=1, TN=27). High recall / low precision: the static heuristic rarely misses true superlinear behaviour but over-flags it — the expected trade-off for a conservative AST rule.

Table 13. Agreement matrix, static estimator vs empirical slope ($n=61$, precision=0.212, recall=0.875).

	Empirical slope < 1.5	Empirical slope ≥ 1.5
Static rank < 2	27	1
Static rank ≥ 2	26	7

Reliable-slope probe (large-scale ≥ 5 ms): 73 program slopes across 14 reference tasks; median candidate slope 1.0218, median reference 1.1192. Reliable medians — candidate 1.02 vs reference 1.12 — confirm the harness scales near-linearly and the probe is calibrated.

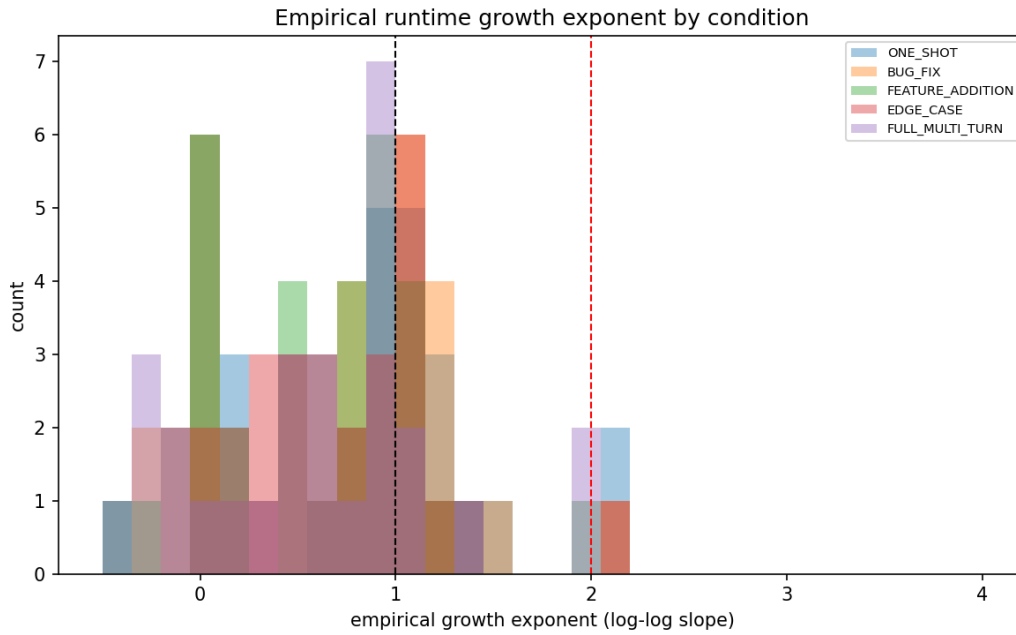


Figure 10. Empirical growth-exponent (log-log slope) distribution. Most programs scale near-linearly; the superlinear tail is small.

4.10 Complexity, size, and energy (RQ5)

Table 14. Cross-metric correlations on measured programs (from `complexity_summary.json`).

Pair	n	Spearman ρ	Pearson r
SLOC vs energy	1309	0.0388	0.0756
Cyclomatic vs energy	1309	-0.0755	0.0571
Complexity rank vs energy	1309	-0.1539	0.0402
Loop depth vs energy	1309	-0.2869	-0.0373
SLOC vs runtime	1309	0.0069	0.0757
Runtime vs energy	1309	0.9325	0.9952

Code-level complexity correlates weakly with energy across heterogeneous tasks, whereas runtime correlates almost perfectly. The energy penalty of multi-turn code is transmitted primarily through longer or heavier executions, not through source-level complexity per se.

4.11 Carbon-equivalent emissions

The measured programs consumed 1853.011 J in total — 247.0681 mgCO₂eq at the default global-average intensity (480 g/kWh). Per-run multi-turn overhead versus one-shot is a few $\times 10$ μ gCO₂eq at this intensity — negligible per run, material at datacentre scale.

Table 15. Carbon sensitivity by grid intensity (from `carbon_report.json`).

Grid	gCO ₂ eq/kWh	Median C0 μ g	Median C4 μ g	Total mg
global_average	480	80.72	107.32	247.068107
us_average	369	62.053	82.502	189.933607
eu_average	251	42.21	56.119	129.196031
low_carbon_france	56	9.417	12.521	28.824612

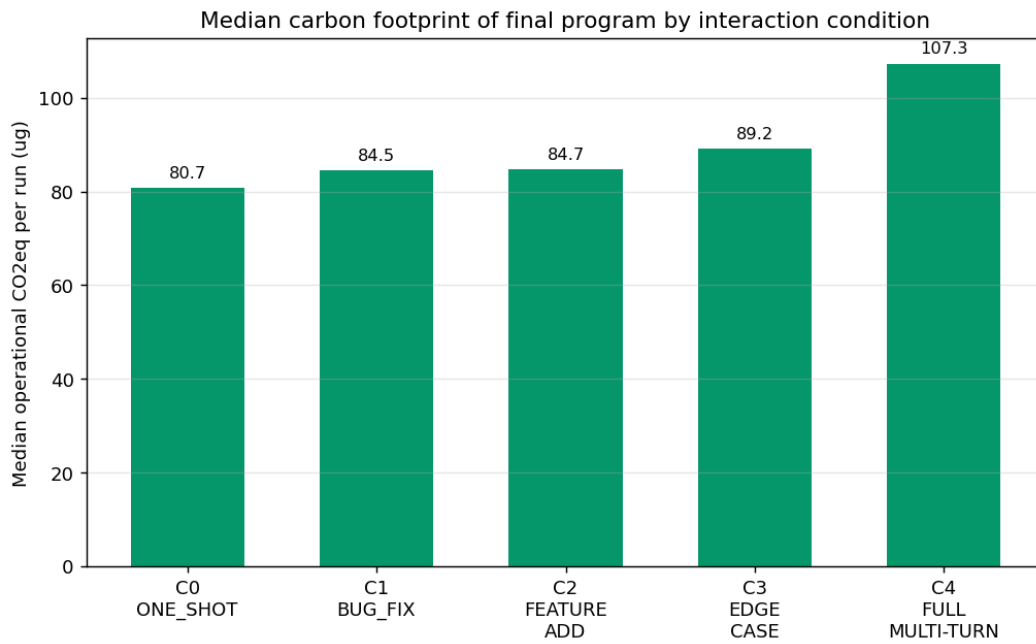


Figure 11. Carbon-equivalent by condition across grid intensities. Rankings are intensity-invariant.

4.12 Failure taxonomy and coverage bias

176 units remain unmeasured. The largest class is WRONG_OUTPUT (78) — programs whose embedded self-tests fail — followed by runtime bugs and CLI-argument mismatches; failures concentrate in the algorithms category. Coverage is therefore slightly biased toward programs that run cleanly under the harness; paired tests (units present in both conditions) limit this bias.

Table 16. Unmeasured units by reason (from energy_failures.csv).

Failure reason	Units
WRONG_OUTPUT	78
RUNTIME_BUG	33
CLI_ARGS	20
NOT_PYTHON	14
SILENT_RC1	12
OTHER	11
MISSING_DEP	5
MISSING_FILE	3

Table 17. Unmeasured units by category.

Category	Unmeasured units
Algorithms & Computation (AC)	93
File & Data Processing (FD)	34
Image & Media Processing (IM)	33
Realistic Applications & Utilities (RA)	10
Text & Log Processing (TL)	6

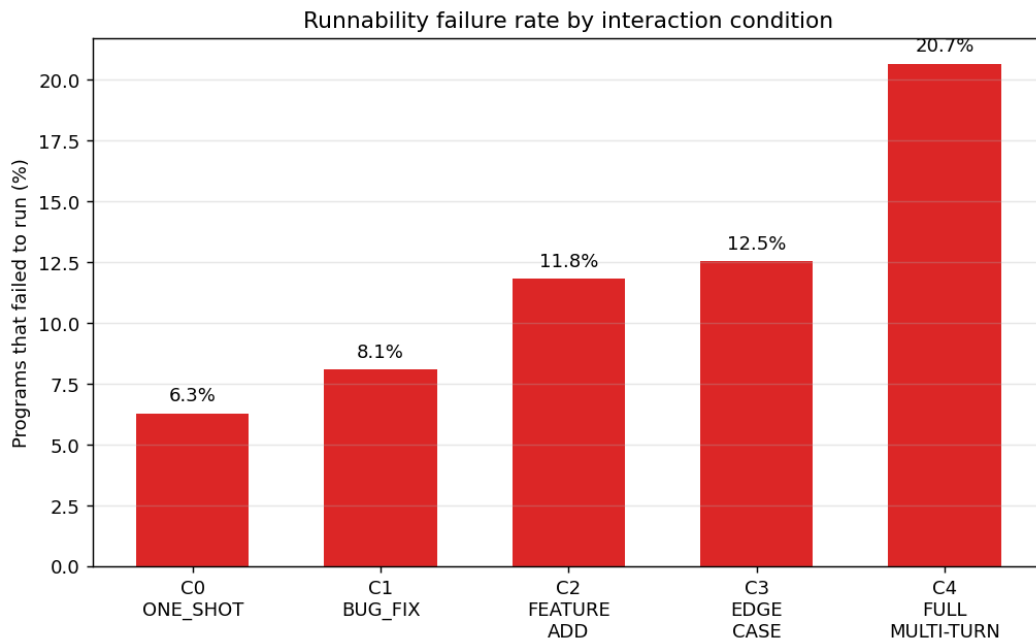


Figure 12. Unmeasured-unit rate by condition. Full multi-turn fails most often — complexity has a correctness cost too.

5 Discussion

5.1 Interpretation

Multi-turn interaction leaves a measurable green-software footprint. The median program barely changes between one-shot and bug-fix, but the full trajectory shifts the median upward and, more importantly, thickens the upper tail: the mean rises far more than the median. Practically, most sessions cost little extra — a minority cost a lot.

5.2 Why multi-turn programs consume more energy

- Accumulated requirements produce longer programs, and longer code tends to encode richer, less pruned algorithms (+57 mean / +32 median SLOC C0→C4).
- Edge-case and feature turns encourage defensive branches and additional passes over the input (cyclomatic 14→18).
- Some trajectories converge on asymptotically heavier solutions, visible as superlinear empirical slopes (C4 reliable-slope superlinear rate 18% vs 6% overall).

5.3 Complexity as a mediator

Static complexity rises with interaction depth, but its direct correlation with energy is weak across categories. The mediating variable is runtime: $\text{energy} \approx \text{power} \times \text{time}$, and runtime tracks the input-scaling behaviour of the chosen algorithm. Treat algorithmic-complexity regressions — not raw SLOC — as the leverage point.

5.4 Implications for practitioners

- Treat iterative features as potential complexity regressions and re-benchmark the final artifact, not just the first draft.
- Ask assistants to justify asymptotic cost after each interactive turn, not only at the end.
- Terminate lengthening sessions with a complexity review; prefer pruning passes before merging.

5.5 Implications for tool builders

IDE-integrated green linters could surface estimated complexity and measured energy deltas per turn, making the accumulating cost visible during interactive development — e.g. flagging a turn that moves estimated class from $O(n \log n)$ to $O(n^2)$.

5.6 Carbon scope and scale

Scope: operational carbon from CPU package energy only — a lower bound excluding DRAM/GPU, idle power, embodied carbon, and LLM inference energy. Scale (illustrative, global-average grid): the paired mean overhead ($+0.716 \text{ J} \approx +95.5 \mu\text{g/run}$) reaches $\approx 95.5 \text{ g}$ per million and $\approx 95.5 \text{ kg}$ per billion executions ($\approx 620 \text{ km}$ of driving; ≈ 5 tree-years); the paired median ($+0.0315 \text{ J} \approx +4.2 \mu\text{g}$) reaches $\approx 4.2 \text{ kg}$ per billion. The worst corpus program (AC-012, deepseek edge-case, 67.9 J) emits $\approx 9.1 \text{ mg}$ per run — $85\times$ the median — so fleets are driven by tails. Real-world impact is amplified by rework waste (20.7% vs 6.3% failure) and rebound effects, but the near-zero bug-fix turn (median -0.1%) shows not all iteration is harmful.

6 Threats to validity

6.1 Construct validity

RAPL package energy includes components beyond the program under study; identical workloads and a per-run baseline mitigate but do not eliminate noise. The static Big-O estimator is an explicit heuristic, cross-checked both ways (declared targets, empirical slopes).

6.2 Internal validity

Harness workloads are deterministic and shared across conditions; paired within-(task, model) tests control for task difficulty. The 176 unmeasured units bias coverage toward runnable programs.

6.3 External validity

Results come from one host (Intel i5-8250U, package RAPL), Python only, and four models. Generalisation to other hardware, models, or languages requires replication. Search & Retrieval ($n=2$) cannot support category-level claims.

6.4 Conclusion validity

Only C4 reaches significance; single-step claims are withheld. The empirical probe covers the algorithms category (190 programs); static metrics cover all 1485. Reliable-slope filtering ($\geq 5 \text{ ms}$) excludes overhead-dominated fits.

7 Conclusion and future work

7.1 Summary

We introduced a benchmark isolating the effect of human-AI interaction trajectory on the energy of final programs. Holding final specification constant, full multi-turn trajectories increase median package energy relative to one-shot generation (the only statistically significant effect), mediated by runtime and algorithm choice rather than source size alone. We release a per-program corpus of static metrics and dual (static + empirical) time-complexity estimates with full reproduction scripts.

7.2 Future work

- Extension to more models, languages, and hardware (including non-RAPL power measurement).

- Per-turn energy attribution across entire sessions.
- Automatic complexity-regression detection during interactive coding.
- Larger scaling sweeps with finer input-size grids and all six categories.

Data availability

Per-program corpus results/final/code_metrics.csv (1485 rows); tables complexity_metrics.json / energy_report.json / carbon_report.json; slopes scaling.csv / scaling_summary.json; figures results/final/plots/ (16 PNGs); reproduction commands in Appendix E. Energy ledgers results/energy_runs.jsonl and results/measurement_ledger.jsonl.

Acknowledgments

Compute and measurement host providers; maintainers of Intel RAPL tooling, python-docx, and LibreOffice for document conversion. [Add funding / grant acknowledgments before submission.]

Author contributions

All authors contributed equally to task design, interaction design, measurement, analysis, and writing.

References

- [1] E. Rotem, A. Naveh, D. Rajwan, A. Ananthakrishnan, and E. Weissmann. Power-management architecture of the Intel microarchitecture code-named Sandy Bridge. *IEEE Micro*, 32(2):20–27, 2012.
- [2] M. Hähnel, B. Döbel, M. Völp, and H. Härtig. Measuring energy consumption for short code paths using RAPL. *ACM SIGMETRICS Performance Evaluation Review*, 40(3):13–17, 2012.
- [3] D. Hackenberg et al. An energy efficiency feature survey of the Intel Haswell processor. *Proc. IPDPSW*, 2013.
- [4] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.
- [5] D. Patterson et al. Carbon emissions and large neural network training. *arXiv:2104.10350*, 2021.
- [6] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in NLP. *Proc. ACL*, 2019.
- [7] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, and Y. Zou. Green AI: Do deep learning frameworks have different costs? *Empirical Software Engineering*, 27:1–38, 2022.
- [8] A. Hindle. Green software engineering: The curse of methodology. *Empirical Software Engineering*, 2023.
- [9] G. Pinto, F. Castor, and Y. D. Liu. Understanding energy behaviors of thread management constructs. *Proc. OOPSLA*, 2014.
- [10] R. Pereira et al. Energy efficiency across programming languages: How do energy, time, and memory relate? *Proc. SLE*, 2017.
- [11] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021 (Codex / HumanEval).
- [12] J. Austin et al. Program synthesis with large language models. *arXiv:2108.07732*, 2021 (MBPP).
- [13] R. Nijkamp et al. CodeGen: An open large language model for code with multi-turn program synthesis. *Proc. ICLR*, 2023.
- [14] T. McCabe. A complexity measure. *IEEE Trans. Software Engineering*, SE-2(4):308–320, 1976.
- [15] C. C. S. L. Pinto, F. Castor, and L. Maia. Energy efficiency: A new concern for software engineers. *Proc. SAC*, 2014.
- [16] J. Schmidt et al. CodeCarbon: Estimate and track carbon emissions from machine learning computing. *GitHub*, 2021.
- [17] A. Anwar et al. Datacenter-scale energy and carbon accounting. *Proc. HotCarbon*, 2022.
- [18] H. Li et al. SWE-bench: Can language models resolve real-world GitHub issues? *Proc. ICLR*, 2024.
- [19] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *Proc. NeurIPS*, 2024.
- [20] B. Rozière et al. Code Llama: Open foundation models for code. *arXiv:2308.12950*, 2023.
- [21] D. Guo et al. DeepSeek-Coder: When the large language model meets programming. *arXiv:2401.14196*, 2024.
- [22] Anthropic. Claude 3.5 Sonnet model card. 2024.
- [23] Google DeepMind. Gemini 1.5 model card. 2024.
- [24] OpenAI. GPT-4o system card. 2024.

- [25] D. Huang et al. EffiBench: Benchmarking the efficiency of automatically generated code. Proc. NeurIPS, 2024.
- [26] A. Di Bernardo et al. Do LLMs dream of energy-efficient code? Proc. LLM4Code (ICSE Workshop), 2026.

Appendix A: Task catalog

Each category provided up to 25 candidate tasks with a declared complexity target where applicable.

Table 18. Full task catalog (from dataset/*/dataset.json).

Cat	Task	Title	Difficulty	Declared
AL	AC-001	Interval Conflict Analyzer	medium	$O(n \log n)$
AL	AC-002	Dependency Build Planner	medium	$O((V+E) \log V)$
AL	AC-003	Single-Source Route Analyzer	medium	$O((V+E) \log V)$
AL	AC-004	Weighted Job Scheduler	medium	$O(n \log n)$
AL	AC-005	Longest Increasing Route	medium	$O(n^2)$
AL	AC-006	Largest Threshold Region	medium	$O(RC)$
AL	AC-007	Minimum Spanning Forest	medium	$O(E \log E)$
AL	AC-008	Mutable Range Sum Engine	medium	$O((n+q) \log n)$
AL	AC-009	Minimum Coin Construction	medium	$O(\text{target} * \text{number_of_coins})$
AL	AC-010	Matrix Chain Planner	medium	$O(n^3)$
AL	AC-011	Capacity-Constrained Knapsack	medium	$O(nC)$
AL	AC-012	Longest Common Subsequence	medium	$O(nm)$
AL	AC-013	Rectangle Statistics Engine	medium	$O(RC + \text{sum}(\text{query area}))$
AL	AC-014	One-Dimensional K-Cluster Partitio	medium	$O(kn^2)$
AL	AC-015	Maximum Flow Planner	medium	$O(V^2E)$ worst-case
AL	AC-016	Priority Queue Simulator	medium	$O(n \log n)$
AL	AC-017	Prime Range Analyzer	medium	Approximately $O((R-L) \log \log R)$
AL	AC-018	Exact Polynomial Product	medium	$O(nm)$
AL	AC-019	Minimum-Cost Grid Path	medium	$O(RC)$
AL	AC-020	Traveling Salesperson Solver	large	$O(n^2 2^n)$
AL	AC-021	Minimum-Cardinality Subset Sum	medium	$O(nT)$
AL	AC-022	Edit Distance With Transposition	medium	$O(nm)$
AL	AC-023	Threshold Connectivity Tracker	medium	$O((E+Q) \log E)$
AL	AC-024	Minimum Room Assignment	medium	$O(n \log n)$
AL	AC-025	Multi-Stage Resource Allocation	medium	$O(\text{stages} * \text{options} * \text{budget} * \text{memory})$
FI	FD-001	Latest customer event deduplicatio	medium	-
FI	FD-002	Nested NDJSON request statistics	medium	-
FI	FD-003	Sales region category aggregation	easy	-
FI	FD-004	Inventory snapshot reconciliation	medium	-
FI	FD-005	Timestamped log ordering	hard	-
FI	FD-006	Customer record normalization	medium	-
FI	FD-007	User session interval merger	medium	-
FI	FD-008	Duplicate text file fingerprint in	hard	-
FI	FD-009	Rolling transaction anomaly summar	hard	-
FI	FD-010	Nested JSON inventory flattening	medium	-
FI	FD-011	Time-series gap filling	medium	-
FI	FD-012	Word frequency across text files	medium	-
FI	FD-013	CSV column statistics	easy	-
FI	FD-014	Record linkage with blocking	hard	-
FI	FD-015	JSONL sort-filter pipeline	medium	-
FI	FD-016	Multi-file sales merge join	hard	-
FI	FD-017	Event type pivot table	medium	-
FI	FD-018	Duplicate normalized records	hard	-
FI	FD-019	File manifest integrity report	medium	-
FI	FD-020	Log severity window counter	medium	-
FI	FD-021	JSON configuration merger	medium	-
FI	FD-022	Fixed-width parser exporter	easy	-
FI	FD-023	Directed relationship aggregator	hard	-
FI	FD-024	Hierarchical path usage	medium	-
FI	FD-025	Dataset quality report	hard	-
IM	IM-001	Batch Image Resizer with Fixed Tar	easy	-
IM	IM-002	Recursive Thumbnail Cache Generato	easy	-
IM	IM-003	Bulk Image Format Converter (PNG <	easy	-
IM	IM-004	Grayscale and Pixel Normalization	medium	-
IM	IM-005	Configurable Batch Gaussian Blur	medium	-
IM	IM-006	Image Metadata Inventory Builder	easy	-
IM	IM-007	Perceptual Average-Hash	medium	-

Cat	Task	Title	Difficulty	Declared
		Image Simi		
IM	IM-008	Exact Duplicate Image Finder via C	easy	-
IM	IM-009	Per-Image Histogram and Brightness	medium	-
IM	IM-010	Batch Rotate-and-Center-Crop Pipel	medium	-
IM	IM-011	Media Organizer: Sort Images by Re	easy	-
IM	IM-012	Letterbox Resize (Aspect-Preservin	medium	-
IM	IM-013	Contact Sheet (Thumbnail Grid) Com	medium	-
IM	IM-014	Bulk Lossy Re-encode with Quality/	medium	-
IM	IM-015	RGB Channel Splitter and Recombine	medium	-
IM	IM-016	Sobel Edge Magnitude Filter	hard	-
IM	IM-017	Custom Text-Tag Embedding and Extr	medium	-
IM	IM-018	Near-Duplicate Detection via Histo	medium	-
IM	IM-019	Fuzzy Duplicate Detection via Aver	hard	-
IM	IM-020	Per-Channel Statistics and Brightn	medium	-
IM	IM-021	Batch Semi-Transparent Watermark S	medium	-
IM	IM-022	Deduplicate and Content-Hash Renam	medium	-
IM	IM-023	Responsive Multi-Resolution Image	medium	-
IM	IM-024	Batch Median Filter Denoiser	medium	-
IM	IM-025	Fixed-Size Patch (Tile) Extractor	medium	-
RE	RA-001	Household Expense Analyzer	easy	CPU+memory: parsing, normalization, filtering, and dictionary aggregation.
RE	RA-002	Inventory Reconciliation Utility	easy	CPU+memory: hash-based aggregation over product and sales records.
RE	RA-003	Student Performance Analyzer	easy	CPU-bound aggregation and sorting.
RE	RA-004	Offline Document Organizer	easy	CPU+memory with string processing and sorting.
RE	RA-005	Application Log Summarizer	easy	CPU-intensive parsing and grouping.
RE	RA-006	Configuration Validator	medium	CPU+memory: recursive traversal and validation.
RE	RA-007	Invoice Batch Processor	medium	CPU-bound nested record processing.
RE	RA-008	Contact Search and Duplicate Detec	easy	Mixed CPU+memory indexing workload.
RE	RA-009	Offline Library Catalog Analyzer	easy	CPU+memory aggregation and search.
RE	RA-010	Meeting Room Scheduler	medium	CPU-intensive sorting and heap scheduling.
RE	RA-011	Backup Manifest Comparator	medium	Memory-intensive indexing plus CPU comparison.
RE	RA-012	Transaction Categorizer	medium	CPU-intensive rule matching; $O(\text{transactions} \times \text{rules} \times \text{keywords})$ baseline.
RE	RA-013	Local Sales Report Generator	medium	CPU+memory aggregation and sorting.
RE	RA-014	Offline Product Recommender	medium	CPU+memory: sparse similarity computation.
RE	RA-015	Attendance Analytics Utility	medium	CPU+memory event sorting and state tracking.
RE	RA-016	Local Document Keyword Index	medium	Memory-intensive inverted indexing plus CPU query ranking.
RE	RA-017	Customer Record Deduplicator	medium	Memory-intensive normalization and union-style grouping.
RE	RA-018	Utility Meter Anomaly Reporter	hard	CPU+memory-intensive sorting and per-meter statistics.
RE	RA-019	Delivery Route Planner	hard	CPU-intensive nearest-neighbor selection with matrix memory.
RE	RA-020	Local File Duplicate Analyzer	hard	I/O-like byte processing plus CPU hashing and memory for content.
RE	RA-021	Personal Schedule Conflict Analyze	hard	CPU-intensive interval processing; naive conflict enumeration may be quadratic in dense schedules.

Cat	Task	Title	Difficulty	Declared
RE	RA-022	Configuration Migration Utility	hard	Mixed CPU+memory recursive transformation.
RE	RA-023	Cash Flow Forecast Utility	hard	CPU+memory date generation and aggregation.
RE	RA-024	Dataset Quality Profiler	hard	Memory-intensive multi-field scanning and hashing.
RE	RA-025	Archive Manifest Builder	hard	Mixed byte-processing, hashing, string processing, and aggregation.
SE	SR-001	Inverted Index Keyword Search Engi	medium	-
SE	SR-002	Prefix Search over Product Names u	medium	-
SE	SR-003	Large-Scale Exact Record Lookup by	hard	-
SE	SR-004	Multi-Field Structured Record Filt	medium	-
SE	SR-005	Fuzzy Local Matching with Bounded	medium	-
SE	SR-006	Duplicate Customer Record Matching	hard	-
SE	SR-007	Ranked Document Retrieval using TF	medium	-
SE	SR-008	Boolean Query Processor (AND / OR	medium	-
SE	SR-009	On-Disk-Style Local Index Construc	hard	-
SE	SR-010	BM25-Style Ranked Retrieval	medium	-
SE	SR-011	Metadata Tag Search over Media Lib	medium	-
SE	SR-012	Batch Processing of Multiple Indep	hard	-
SE	SR-013	Numeric Range Query via Sorted Ind	medium	-
SE	SR-014	Multi-Document Substring Search	medium	-
SE	SR-015	Autocomplete Suggestion Ranking	hard	-
SE	SR-016	Near-Duplicate Document Detection	medium	-
SE	SR-017	Multi-Keyword Search with Mixed AN	medium	-
SE	SR-018	Application Log Search by Timestam	hard	-
SE	SR-019	E-Commerce Product Catalog Search	medium	-
SE	SR-020	Spelling Correction Candidate Retr	medium	-
SE	SR-021	Grid-Bucketed Nearest Point Search	hard	-
SE	SR-022	Contact Directory Reverse Lookup b	medium	-
SE	SR-023	Email Domain Grouping and Domain S	medium	-
SE	SR-024	Citation Reference Matching Across	hard	-
SE	SR-025	Word Frequency Ranked Search Index	medium	-
TE	TL-001	Apache Access Log IP Frequency & B	Medium	CPU-intensive regex parsing and sorting, I/O-bound scaling.
TE	TL-002	Multi-File JSON Log Stream Merger	Medium	Memory-intensive sorting and CPU-bound JSON parsing.
TE	TL-003	Regex-Based Security Incident Extr	Medium	REGEX/PATTERN-intensive.
TE	TL-004	Large Text Document Concordance an	Easy	CPU-intensive tokenization and hashing.
TE	TL-005	Duplicate Line Detection and Remov	Medium	MEMORY-intensive hash set operations and REGEX.
TE	TL-006	CSV Log Normalizer and Missing Val	Easy	I/O-intensive string parsing and CPU-bound median calculation.
TE	TL-007	Sliding Window Log Anomaly Detecto	Hard	CPU-intensive sliding window iteration.
TE	TL-008	Structured JSON-Lines Error Code C	Easy	I/O-intensive parsing and CPU-bound aggregation.
TE	TL-009	Email Header Parser and Thread Rec	Medium	REGEX/PATTERN-intensive and CPU-bound parsing.
TE	TL-010	Long-Record Genomic FASTA Sequence	Medium	MEMORY-intensive and CPU-bound string counting.
TE	TL-011	SQL Query Log Parser for Slow-Quer	Medium	REGEX/PATTERN-intensive and CPU-bound aggregation.
TE	TL-012	Multi-Column Log Table Formatter a	Easy	CPU-intensive string formatting and alignment.
TE	TL-013	Streaming Log Keyword Search and C	Medium	I/O-intensive search and MEMORY-bound context slicing.

Cat	Task	Title	Difficulty	Declared
TE	TL-014	HTTP Request Session Reconstructor	Hard	CPU-intensive grouping and sequence hashing.
TE	TL-015	Text Normalization Pipeline with S	Easy	CPU-intensive string manipulation.
TE	TL-016	Continuous Chunked Log File Tokeni	Medium	MEMORY-intensive and CPU-bound sliding window counting.
TE	TL-017	XML/HTML Tag Log Extractor and Att	Medium	REGEX/PATTERN-intensive and CPU-bound.
TE	TL-018	Firewall Packet Filter Log Analyze	Medium	CPU-intensive regex parsing and I/O-bound scaling.
TE	TL-019	Substring Pattern Frequency Matrix	Hard	CPU-intensive matrix construction and string searching.
TE	TL-020	Version Control Commit Log Parser	Medium	CPU-intensive parsing and aggregation.
TE	TL-021	Fixed-Width Record File Parser and	Easy	I/O-intensive string slicing and CPU-bound casting.
TE	TL-022	Nested JSON Configuration Log Flat	Hard	MEMORY-intensive recursive flattening and dictionary diffing.
TE	TL-023	Regex-Heavy Sensitive Data Masker	Hard	REGEX/PATTERN-intensive and CPU-bound substitution.
TE	TL-024	Time-Series Log Interpolator and G	Hard	CPU-intensive numerical interpolation.
TE	TL-025	Hierarchical Directory Text Search	Hard	I/O-intensive virtual search and CPU-bound snippet extraction.

Appendix B: Per-condition static metrics

Table 19. Static metrics per condition (from `complexity_metrics.json`).

Condition	n	SLOC	Loops	Depth	Cyclo.	Rank	%rec	Compl.%
C0	302	58.0	3.0		14.0	1.5	1.7	
C1	297	63.0	3.0		15.0	1.5	3.4	
C2	296	82.0	3.0		15.0	1.5	3.8	
C3	295	80.0	3.0		16.0	1.5	5.1	
C4	295	98.5	3.0		18.0	1.5	3.1	

Appendix C: Metric glossary

Table 20. Glossary of tracked metrics.

Term	Meaning
Package energy	CPU-socket energy from RAPL (J/run)
Core energy	CPU core-domain energy from RAPL
ΔE	$(E_{\text{condition}} - E_{\text{one_shot}}) / E_{\text{one_shot}} \times 100$, paired within (task, model)
Power	Package energy / runtime (W)
SLOC	Non-blank, non-comment source lines
Cyclomatic	McCabe decision-point count + 1
Complexity rank	Ordinal of estimated Big-O class
Empirical slope b	$\log(\text{runtime})$ vs $\log(\text{input-bytes})$ exponent
Compliance	Estimated class no worse than declared target
Reliable slope	Large-scale runtime ≥ 5 ms (excludes overhead-dominated fits)

Appendix D: Failure taxonomy

Table 21. Failure taxonomy (from `energy_failures.csv`).

Failure kind	Units	Meaning	Input-fixable
WRONG_OUTPUT	78	Embedded self-test assertion fails	No
RUNTIME_BUG	33	Raises at runtime	Sometimes
CLI_ARGS	20	No invocation produced exit 0	Sometimes
NOT_PYTHON	14	Prose/truncated, not Python	No
SILENT_RC1	12	Non-zero exit without output	Sometimes
OTHER	11	Other launch failure	Sometimes
MISSING_DEP	5	Unavailable third-party import	No
MISSING_FILE	3	Reads a file outside workload	Yes

Appendix E: Reproduction commands

```
python3 scripts/ingest_zips.py zip_src/*.zip
```

```
python3 scripts/gen_inputs.py && python3 scripts/gen_inputs_extra.py
```

```
python3 scripts/measure_energy.py
```

```
python3 analysis/code_metrics.py
```

```
python3 analysis/scaling_probe.py algorithms_computation (extend: --limit N <category> ...)
```

```
python3 analysis/complexity_report.py
```

python3 analysis/energy_report.py

python3 analysis/carbon_report.py

python3 analysis/energy_plots.py

python3 scripts/make_paper_pro.py (this paper)

libreoffice --headless --convert-to pdf docx/green-code-interaction-benchmark.docx --outdir docx/

Index

Static subject index (generated; see `scripts/add_static_index.py`)

Bug-fix condition (C1) — §3.3, §4.3

Carbon-equivalent emissions — §3.11, §4.11

Code metrics corpus (`code_metrics.csv`) — §3.8, §4.1, App. B-C

Complexity rank — §3.8–3.9, §4.10, App. C

Cyclomatic complexity — §3.8, §4.7, §4.10

Dataset coverage (1485 finals, 1309 measured) — §4.1

Edge-case condition (C3) — §3.3, §4.3

Empirical scaling probe (log-log slope b) — §3.10, §4.9

Energy measurement, Intel RAPL — §3.6, §4.2

Failure taxonomy (WRONG_OUTPUT, RUNTIME_BUG, ...) — §4.12, App. D

Feature-addition condition (C2) — §3.3, §4.3

Full multi-turn condition (C4) — §3.3, §4.2–4.5

Harness protocol (correctness vs energy) — §3.5

Heavy-tail effect (mean vs median) — §4.2–4.3, §5.1

Interaction trajectory — §1–§5

Log-log slope, reliable (≥ 5 ms) — §3.10, §3.12, §4.9, App. C

Models (GPT, Claude, Gemini, DeepSeek) — §3.4, §4.4

One-shot condition (C0) — §3.3, §4.2–4.3

Package energy — §3.6, App. C

Paired analysis (ΔE), Wilcoxon signed-rank — §3.12, §4.3

Power (energy/runtime) — §3.7, §4.6

Reproduction commands — App. E

Runtime vs energy (Spearman $\rho = 0.93$) — §4.6, §4.10

SLOC (source lines of code) — §3.8, §4.7, §4.10

Static Big-O estimator (AST heuristic) — §3.9, §4.8–4.9

Task taxonomy (6 categories $\times$ 25 tasks) — §3.2, App. A

Threats to validity — §6

Time complexity, static vs empirical — §3.9–3.10, §4.8–4.9